A Three-Layer Hierarchical Intrusion Detection System for IoT Networks

OLLSCOIL NA GAILLIMHE
UNIVERSITY OF GALWAY

Bolun Liu
School of Computer Science
University of Galway

Supervisor(s)
Dr.Waqar Shahid Qureshi

In partial fulfillment of the requirements for the degree of
MSc in Computer Science (Adaptive Cybersecurity)
August 21, 2025

DECLARATION I, Bolun Liu, hereby declare that this thesis, titled "A Three-Layer Hierarchical Intrusion Detection System for IoT Networks", and the work presented in it are entirely my own except where explicitly stated otherwise in the text, and that this work has not been previously submitted, in part or whole, to any university or institution for any degree, diploma, or other qualification.

Signature: Bolun Liu

Abstract

With the rapid development of the Internet of Things (IoT), the large-scale deployment of terminal devices in scenarios such as smart homes, industrial IoT, and smart cities has made security issues increasingly prominent.Traditional intrusion detection systems (IDS) exhibit limitations in handling resource-constrained environments and detecting unknown attacks.To address this, this study proposes a three-layer modular intrusion detection framework for IoT environments, integrating Autoencoder, XGBoost, and RNN-BiLSTM models, with the aim of balancing detection accuracy, the ability to identify unknown attacks, and resource adaptability.

Experimental results on the Bot-IoT and ToN-IoT datasets demonstrate that the overall detection accuracy of the framework ranges between 92% and 95%. For known attacks, the precision and recall reach 0.99 and 0.95, respectively, while the framework also maintains strong detection performance against unknown attacks, achieving up to 97% accuracy in certain experiments.In addition, the system exhibits low latency (approximately 7 ms), high throughput (around 135 samples per second), and low resource consumption (memory < 450 MB, CPU usage $< 3\%$) in edge device environments, confirming its feasibility and practical value in resource-constrained scenarios.

The main contribution of this paper lies in the proposal of a multi-module hierarchical detection framework that enables efficient iden-

tification of both common and unknown attacks, while maintaining real-time performance and deployability under resource-constrained conditions.However, the study still has certain limitations, such as insufficient effectiveness in detecting complex payload-based attacks, strong reliance on feature engineering, and limited cross-dataset generalization capability.Future research will further explore data augmentation, multimodal fusion, and lightweight optimization methods to enhance the robustness and adaptability of the system.

Keywords: IoT Security; Intrusion Detection System (IDS); Autoencoder; XGBoost; RNN-BiLSTM; Edge Computing

Contents

List of Figures

List of Tables

Chapter 1

Introduction

With the rapid development of the Internet of Things (IoT), an increasing number of smart devices are being integrated into households, industries, healthcare systems, and urban infrastructures.IoT devices have become increasingly prevalent in daily life. Examples include smartwatches widely adopted by individuals, smart lighting systems, fingerprint locks, and surveillance cameras in households, as well as smart vehicles and air quality sensors in urban environments, all of which have grown significantly in recent years.According to recent statistical reports, the number of IoT devices worldwide has grown exponentially over the past decade and is projected to reach tens of billions by 2030[1]. This rapid expansion is largely driven by the convenience offered by IoT technologies across diverse domains. For instance, smart home systems enable remote and automated control, while industrial IoT facilitates efficient and intelligent monitoring. Moreover, the widespread deployment of sensors supports optimization in areas such as transportation and energy management.However, the rapid proliferation of IoT devices has also introduced significant security risks, making IoT systems highly vulnerable to various threats.Most IoT devices are constrained by limited computational power, restricted storage capacity, pronounced system heterogeneity,

and wide-scale distribution, which makes them attractive targets for malicious actors.Adversaries can launch large-scale attacks on IoT devices through methods such as malware propagation and distributed denial-of-service (DDoS) attacks, thereby posing severe threats and disruptions to network environments and related application domains.For example, the Mirai botnet attack, first identified by a white-hat security organization in August 2016, exploited IoT devices to launch a large-scale network assault. This incident incapacitated hundreds of websites, including Twitter, Netflix, Reddit, and GitHub, for several hours[2]. This incident underscores the critical importance of ensuring cybersecurity in modern IoT environments.

Therefore, to address the rapidly growing cybersecurity threats in IoT environments, intrusion detection systems (IDS) have become another major focus of research for cybersecurity scholars.IDS are capable of monitoring network traffic, device behaviors, and device status in real time, and through such monitoring they can identify abnormal activities and potential attacks, thereby ensuring network security and the proper functioning of IoT devices.Existing traditional IDS systems are mostly deployed in the cloud or on centralized servers, relying on high-performance computing resources and abundant storage capacity to execute complex yet high-performance detection algorithms[3]. However, these characteristics make them clearly unsuitable for deployment in large-scale IoT networks, as they exhibit several inherent limitations:First, IoT devices generate massive amounts of data that require highly real-time processing.Second, transmitting IoT data over the network (e.g., uploading to the cloud) increases the risk of data exposure, particularly for IoT devices used in sensitive domains such as healthcare and industrial control.Finally, continuously uploading data to the cloud or central servers increases power consumption of IoT devices and constantly occupies bandwidth and other resources.These issues represent significant drawbacks

when applying such IDS systems to resource-constrained edge devices.Therefore, lightweight and efficient IDS solutions, suitable for deployment on IoT edge devices, become particularly important[4].

In recent years, researchers have employed various approaches to improve the performance of IoT intrusion detection systems (IDS).For example, techniques such as feature selection and dimensionality reduction have been used to decrease model complexity and enhance performance.Lightweight neural networks combined with model pruning methods have been applied to reduce computational overhead, making IDS more suitable for IoT environments.Furthermore, online learning and incremental learning strategies have been integrated to strengthen IDS capabilities in detecting dynamically evolving data.Other studies have explored collaborative architectures that integrate cloud servers with edge computing to balance real-time processing and detection accuracy, thereby adapting IDS to IoT environments.However, current IDS solutions designed for IoT edge devices still face several unresolved challenges:first, inference speed and power consumption still require further optimization;second, their ability to detect zero-day or previously unseen attacks remains insufficient or inconsistent.The persistence of these challenges highlights the need for continued research and exploration in this field.

In response to the aforementioned issues, this study aims to propose an optimized IDS model capable of detecting unknown attacks and suitable for IoT edge devices.

1.1 Research Questions

RQ1: How can a lightweight intrusion detection system (IDS) be designed to operate efficiently on resource-constrained IoT edge devices?

RQ2: To what extent can feature selection and model optimization techniques improve the detection accuracy of IoT IDS while maintaining the false positive rate within an acceptable range?

RQ3: When deployed in a simulated IoT edge device environment, how does the proposed IDS perform in terms of latency, resource utilization, and operational efficiency?

1.2 Main Contributions of this Study

This study aims to ensure the security of IoT devices while achieving IDS usability, flexibility, and scalability.

main contributions of this study can be summarized as follows:

1.Lightweight IDS design: A lightweight IDS model tailored for IoT edge devices is proposed, incorporating hierarchical detection, batch data processing, and delayed loading mechanisms to reduce computational cost and memory usage.

2. Performance optimization strategies: While maintaining detection accuracy, the model achieves high inference speed and low resource consumption, enabling efficient operation on resource-limited edge devices.

3. Experimental validation and evaluation: Extensive experiments on multiple IoT datasets and simulated edge environments demonstrate the model's strong accuracy and superior ability to detect unknown attacks.

4. Comparison with traditional ML models: Based on the ToN-IoT dataset, the proposed IDS is compared against traditional machine learning algorithms, highlighting its performance advantages.

1.3 Structure of the Thesis

The structure of this thesis is organized as follows:Chapter 2 presents a literature review, summarizing existing IDS techniques, the security challenges faced by IoT networks, and prior efforts and achievements in designing IDS solutions for IoT devices;Chapter 3 outlines the methodology, providing a detailed description of the proposed lightweight IDS model, optimization strategies, and corresponding experimental design;Chapter 4 reports the experiments and results, presenting the performance metrics and classification outcomes of the proposed IDS model on multiple datasets (BoT-IoT and ToN-IoT);Chapter 5 provides analysis and discussion of the experimental results, offering insights into their implications and potential directions for improvement;Finally, Chapter 6 concludes the study and outlines possible directions for future research.

Chapter 2

Literature Review

In recent years, the rapid development of Internet of Things (IoT) technology has led to its widespread deployment and application worldwide.From smart cities and smart homes to industrial control systems and healthcare devices, IoT has been deeply embedded into all aspects of human society.According to multiple research sources, the number of IoT devices is growing exponentially, which not only drives improvements in production efficiency and the transformation and upgrading of economic structures, but also accelerates the process of social informatization[5]. However, as the IoT ecosystem continues to expand, its security issues have become increasingly prominent.IoT devices generally possess limited computing resources, diverse communication protocols, and complex operating environments, which, while providing flexibility, also create more opportunities for cyber attackers.In particular, when facing threats such as unknown attacks, zero-day vulnerabilities, and botnets, traditional security protection mechanisms often fail to meet requirements[6].

Against this backdrop, Intrusion Detection Systems (IDS) have attracted widespread attention from both academia and industry as an essential component of IoT security protection[7]. Researchers have explored IoT security threats from mul-

tiple perspectives, ranging from classical pattern-matching approaches, to intelligent detection techniques incorporating machine learning and deep learning, and further to hybrid architectures that combine the strengths of various methods, achieving varying degrees of success and progress.The following will systematically review and analyze the current state of IoT intrusion detection research in recent years from four perspectives: traditional machine learning, unsupervised learning, deep learning, and hybrid approaches.

With the current vigorous development of IoT, IoT devices have gradually spread to every corner of the world, permeating various areas such as daily life, industrial production, smart cities, and healthcare.According to Vivar's research, the development of IoT is closely related to a nation's economic and technological level, and its rapid adoption can positively stimulate economic growth on multiple fronts[8]. However, alongside the enormous opportunities it brings, IoT's security risks have become increasingly evident, particularly regarding the cybersecurity challenges faced by devices.These issues not only threaten personal privacy and information security but may also seriously impact the stable operation of critical infrastructure, thus becoming a key focus for researchers.

Regarding IoT security protection, Hassan et al[9]. found in 2019 that current security measures primarily aim to safeguard privacy and confidentiality, and on that basis, ensure the overall security of data, devices, and infrastructure.However, they pointed out that such security strategies are often formulated based on past, well-known threat and attack cases.Consequently, when confronting unknown attack types or emerging attack patterns, the effectiveness of these measures may be greatly diminished.Based on this analysis, Hassan concluded that future IoT cybersecurity systems should emphasize both the defense against known attacks and the prevention of unknown ones, thereby enhancing overall protection capability and system resilience.

Against this background, with machine learning achieving remarkable success across various industries, researchers have begun introducing it into the field of IoT security for detecting both known and unknown attacks.The Ioannou [10] team applied the Support Vector Machine (SVM) model to IoT network environments, identifying suspicious traffic by analyzing the characteristics of normal network activity.They conducted classification experiments for benign and malicious traffic using two models: C-SVM and OC-SVM.In tests on data with unknown network topologies, the detection accuracy of these models was 81% for C-SVM and 58% for OC-SVM.These results indicate that although SVM has a certain recognition capability in known environments, its detection performance still has considerable room for improvement in complex, dynamic, and topology-unknown IoT environments.

In contrast, Abbade [11] and colleagues opted to use the XGBoost model to mine potential feature patterns from large datasets, aiming to detect replication attacks in IoT environments.In their study, the advantages of XGBoost were fully leveraged: the model not only achieved relatively accurate classification results using labeled data, but also required no additional normalization for numerical features, thereby reducing data preprocessing complexity in practical applications.They further employed a grid search approach to identify optimal training parameters, ultimately achieving a detection accuracy of 93.6% on the IoT-23 dataset, fully demonstrating the superiority of tree-based machine learning methods in detecting known attacks.

However, real-world IoT security threats are not limited to known attacks.Due to the difficulty of obtaining labeled samples, unknown attacks remain a persistent and destructive threat in the IoT domain.To address this issue, Nhlapo [12] and colleagues conducted a comparative study of several machine learning models that perform well in IoT environments.They applied Support Vector Ma-

chine (SVM), Naïve Bayes, and XGBoost models to zero-day attack and ransomware detection tasks, and systematically evaluated the detection performance of each model.Experimental results showed that, compared with the tree-based XGBoost model, the detection performance of SVM and Naïve Bayes was relatively poor.This finding further confirms the robustness of tree-based ensemble learning methods in handling complex attack patterns.At the same time, Nhlapo also emphasized that unsupervised learning models have an inherent advantage in unknown attack detection because they do not rely on labeled data for such attacks, and have therefore received widespread attention and application in this field.

In research related to unsupervised learning, Hindy [13] proposed an original and efficient autoencoder model for zero-day attack detection in IoT environments.They evaluated the model using the CICIDS2017 dataset, comparing it with one-class SVM as the baseline method. The autoencoder employs an unsupervised feature learning mechanism, using an encode–decode structure to capture normal behavior patterns in network traffic data.Once the model is trained, failure to successfully reconstruct new network traffic data indicates the potential presence of malicious activity or botnet attacks. The goal of this method is to build an intrusion detection system (IDS) with a high recall rate while keeping the missed detection rate (false negative rate) as low as possible. Experimental results show that this method achieved zero-day attack detection accuracy of 75%–98% on the CICIDS2017 dataset and 89%–99% on the NSL-KDD dataset, demonstrating strong detection performance and generalization ability.

On this basis, Zahoora et al.[14] proposed an attribute learning technique based on a deep contractive autoencoder (CAE) and combined it with a heterogeneous voting ensemble method to enhance detection performance.In the detection of previously unseen zero-day attacks, this method achieved a recall of 0.95 with only

6 false negatives, demonstrating advantages and greater rationality compared to traditional machine learning techniques.

With the widespread application of deep learning across multiple fields, an increasing number of studies have introduced it into IoT intrusion detection.Aldaej[15] constructed an IDS system using deep learning, composed of a recurrent neural network (RNN) and a bidirectional long short-term memory network (Bi-LSTM), and divided the system into cloud and edge components to accommodate the resource constraints of edge devices.Alqahtani[16]proposed an improved hybrid intrusion detection architecture that combines a deep autoencoder with a bidirectional long short-term memory (Bi-LSTM) network, specifically designed for rapid identification of malicious threats in complex IoT environments.The system achieved outstanding results in both detection accuracy and latency reduction, validating the potential and practical value of hybrid architectures in complex application scenarios.

However, from an overall research perspective, existing architectures still have certain shortcomings: they often fail to simultaneously meet the requirements of unknown attack detection capability, lightweight design to accommodate resource-constrained IoT devices, and integration of the two technologies to fully leverage their respective strengths. Therefore, future research urgently needs to develop an intrusion detection system that can incorporate all three features, ensuring detection effectiveness while meeting the practical needs of IoT applications, thus providing a more comprehensive and efficient security solution for IoT environments.

Chapter 3

Methodology

Against the backdrop of the rapid development of the Internet of Things (IoT), effectively detecting network attacks and ensuring system security have become key issues in research and application.Addressing the diverse data and complex traffic characteristics of IoT environments, this paper proposes a three-layer architecture design scheme for an intrusion detection system.The system integrates multiple machine learning and deep learning models, achieving efficient and accurate identification of normal traffic, known attacks, and unknown attacks through layered screening and classification.The methodology section of this paper elaborates on the system architecture design, the models and algorithms employed in each module, optimization strategies, dataset selection and preprocessing methods, as well as experimental setup and evaluation metrics, aiming to provide a practical and feasible technical approach for intrusion detection in IoT environments.

3.1 System Architecture Design Concept and Overall Process

Designing an efficient IoT intrusion detection system first requires careful consideration of the overall architecture, model selection, and processing workflow to achieve seamless coordination among the models within the system.In IoT settings, devices are resource-constrained, data volumes are large, and traffic characteristics are complex; as a result, a single model often struggles to balance detection speed and accuracy. Accordingly, this project proposes and designs a three-layer architecture comprising three stages: rapid processing, unknown attack detection, and classification of known attacks.The architecture proceeds layer by layer, progressively refining data categorization and identification, thereby ensuring efficient system operation while enhancing the detection of unknown attacks.

The first layer is designed to rapidly process the full volume of incoming data. An Autoencoder (AE) model is employed at this stage to perform an initial, fast screening of IoT traffic, classifying it into normal traffic or attack traffic.The design of this layer takes into account the large number of IoT devices, the complexity of network traffic, and the inherent computational limitations of such devices. Therefore, a lightweight Autoencoder model with unsupervised training is adopted. This model not only captures the behavioral characteristics of normal traffic but also efficiently identifies and filters potential attack traffic, thereby reducing the processing burden for subsequent layers.

The second layer is responsible for detecting unknown attacks. At this stage, the XGBoost model is applied to the traffic identified as attack traffic in the first layer, further categorizing it into known attacks and unknown attacks.This layer leverages the superior capability of the XGBoost model in handling complex

nonlinear data, and incorporates a threshold-based mechanism to distinguish between known and anomalous traffic. By doing so, it effectively separates unknown attacks from known ones, thereby providing critical support for the attack type classification performed in the next layer.

The third layer focuses on the fine-grained classification of known attacks. For the traffic instances identified as known attacks in the previous layers, an RNN-BiLSTM (Recurrent Neural Network with Bidirectional Long Short-Term Memory) model is employed. By combining the temporal modeling capability of RNN with the bidirectional context-capturing strength of BiLSTM, this layer achieves accurate recognition of complex sequential network traffic and categorizes it into specific attack types, such as backdoor and password attacks. Additionally, this layer is capable of detecting normal traffic that may have been mistakenly classified in earlier stages, functioning as a secondary verification step, thereby enhancing the overall detection capability of the framework.

The entire process can be intuitively understood as follows: IoT data flows in and undergoes preliminary normal/attack classification by an autoencoder; attack data then enters the XGBoost module for known/unknown attack division, and the known attack data is further classified into specific attack types by the RNN-BiLSTM module.The system ultimately outputs three categories of results: normal, known attacks, and unknown attacks, greatly enhancing the system's security protection capability and response efficiency.

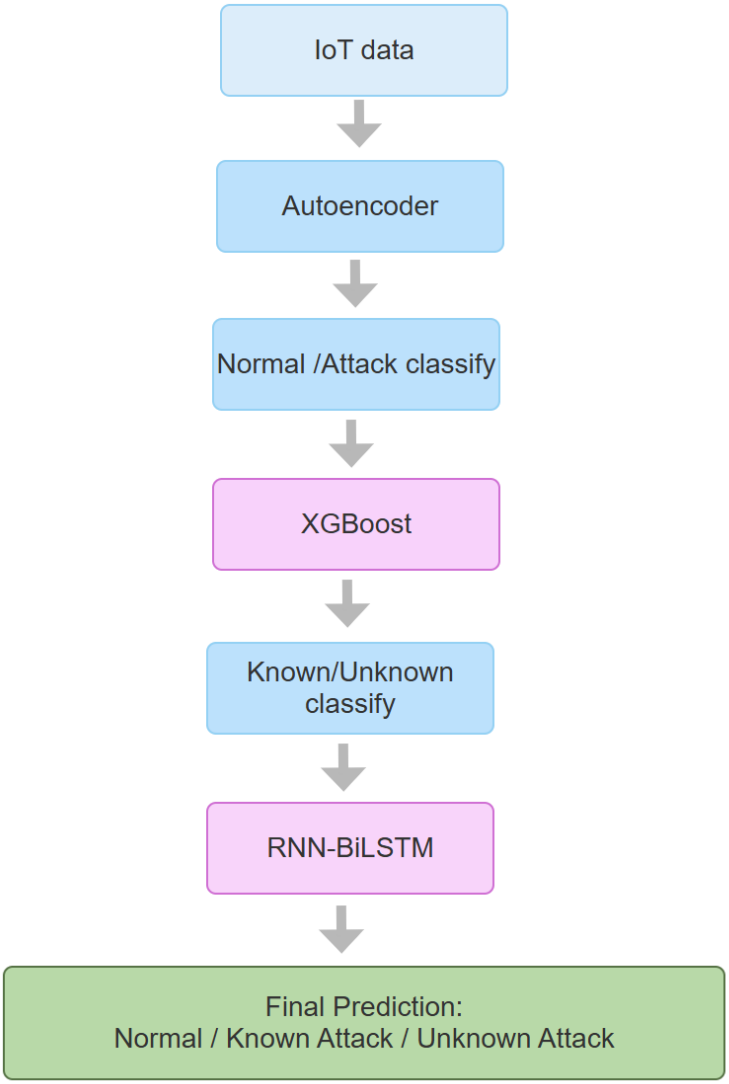

Figure 3.1: IDS System Process

3.2 Model Selection and Module Implementation Details

3.2.1 Preliminary Detection Module: Autoencoder Model

In the Internet of Things (IoT) environment, since a large amount of normal traffic dominates and attack traffic is relatively scarce, using unsupervised learning models to model the normal behavior patterns of data has become an effective strategy.

Autoencoder (AE) is a neural network-based unsupervised learning method that can capture the intrinsic structural features of data by learning the encoding and decoding processes[13].

The basic structure of an autoencoder includes the input layer, encoder, latent space vector, and decoder.The specific process is: input data X1 is transformed into a latent vector Z by the encoder, which is then restored to reconstructed data X2 by the decoder.The goal of model training is to make the reconstructed data as close as possible to the input data, thereby measuring whether the input data deviates from the normal pattern through reconstruction error.A commonly used loss function is Mean Squared Error (MSE) [17]:

$$L = \|x - x'\|^2 \tag{3.1}$$

Here, x represents the input data, and x' represents the reconstructed data.

In this project, the autoencoder is trained only with normal traffic data, allowing the model to learn the characteristics of normal behavior.After training, the full dataset (a mix of normal and attack data) is input into the model to calculate the reconstruction error.Since the model has learned the pattern of normal data, the reconstruction error for normal data is small; attack data usually differs

significantly from normal patterns, resulting in a large reconstruction error.A threshold is set based on the reconstruction error to enable quick differentiation between normal and attack data.

The advantages of the autoencoder are specifically reflected in:

Strong nonlinear modeling capability: able to capture nonlinear patterns of complex data, adapting to the diverse data characteristics of IoT.

Unsupervised learning: does not rely on attack labels, making it suitable for scenarios in IoT where unknown attacks frequently occur.

Simple and efficient structure: suitable for resource-constrained edge devices, supports online incremental training, and enables dynamic updates.

Superior performance: experiments show that the autoencoder significantly outperforms the traditional one-class SVM in recall and F1-score for detecting unknown attacks.

To validate the effectiveness of the autoencoder model, this project compared one-class SVM and traditional SVM models, with results showing:

Model	Accuracy	Recall	Precision	F1-score
Autoencoder	Higher	Higher	Higher	Higher
One-Class SVM	Slightly lower	Lower	Medium	Medium
SVM	Lower	Lower	Medium	Lower

Table 3.1: Comparison of Autoencoder with SVM and One-Class SVM

These results indicate that the autoencoder performs more stably when handling high-dimensional large-sample data in IoT environments, with lower false positive rates and fewer missed detections, making it more suitable for deployment on IoT edge devices.

However, in the IDS architecture of this project, the function of the autoencoder is to classify normal data from attack data. When this functionality is not required in practical deployment scenarios, this layer can be skipped, making the overall

IDS model structure more flexible.

3.2.2 Unknown Attack Detection Module: XGBoost Model

Traditional supervised learning models often rely on large amounts of labeled data and struggle to effectively identify unknown attacks.To enhance the system's capability to detect unknown attacks, this project designed an unknown attack detection module based on XGBoost in the second layer.

XGBoost (Extreme Gradient Boosting) is an ensemble learning method based on gradient-boosted trees, which improves overall model performance by iteratively training multiple decision trees[18]. During each training round, XGBoost fits the residuals (i.e., prediction errors) of the previous model, continuously correcting errors to ultimately obtain a strong classifier.The model update formula is as follows:

$$F_t(x) = F_{t-1}(x) + \eta\, h_t(x) \tag{3.2}$$

Here, $F_t(x)$ represents the model at the $t-th$ iteration, $h_t(x)$ denotes the newly added tree at the $t-th$ iteration, and η is the learning rate.The objective function of XGBoost is:

$$L = \sum_{i=1}^{n} (y_i, \hat{y}_i) + \sum_{t=1}^{T} \Omega(h_t) \tag{3.3}$$

The advantages of XGBoost [19]include:

Integration of multiple decision trees to improve classification accuracy.

Support for GPU parallel computing to enhance training efficiency.

Automatic feature selection to reduce manual intervention.

Regularization and pruning capabilities to effectively prevent overfitting.

Ability to output prediction confidence, facilitating uncertainty analysis.

Good interpretability, which aids in analyzing classification results.

This project utilizes XGBoost's confidence output to divide data initially classified as "attack" into high confidence (known attacks) and low confidence (unknown attacks).During training, only known attack data is used, allowing the model to learn the characteristic behaviors of various attacks.During prediction, samples with confidence below the threshold are considered anomalous, i.e., unknown attacks.

In comparative experiments, XGBoost outperformed multi-class SVM in both accuracy and $F1-score$. SVM showed weaker performance in handling complex nonlinear data, achieving only 74% accuracy and 76% $F1-score$.Therefore, XGBoost becomes the preferred model for unknown attack detection.

3.2.3 Known Attack Classification Module: RNN-BiLSTM Model

Fine-grained classification of known attacks is particularly important for security response and the formulation of defense strategies.IoT network traffic data exhibits obvious temporal characteristics, making it suitable for modeling with Recurrent Neural Networks (RNN)[20].

RNNs capture the dynamic patterns of traffic changes by "remembering" information from previous time steps.Its basic structure consists of multiple recurrent units connected in series, each receiving the current input and the previous hidden state, outputting the current hidden state, with information transmitted along the temporal direction.

However, traditional RNNs suffer from the problems of vanishing gradients and insufficient long-term dependency capture.To address this, Long Short-Term Memory (LSTM) networks, which contain gating mechanisms, are used to effectively solve the vanishing gradient problem and enhance long-term dependency capabil-

ity.By combining Bidirectional LSTM (BiLSTM) technology, both past and future contextual information can be utilized, improving the model's understanding and prediction ability for sequential data.

The RNN-BiLSTM attack classification module designed in this project integrates the advantages of RNN and BiLSTM [21]:

Bidirectional modeling that comprehensively utilizes both past and future temporal information. Lightweight structure suitable for deployment on IoT edge devices. Excellent capability in handling high-dimensional sequential data, adapting to complex IoT traffic environments. Verified through extensive experiments, this module achieves high performance in attack classification accuracy and recall, providing a solid guarantee for overall system performance improvement.

However, due to the nature of sequential learning, the RNN-BiLSTM model requires the training data to exhibit clear temporal features in order to more accurately learn the characteristics of normal and attack data.

3.3 Model Optimization and Parameter Tuning

To further improve system performance, this project implemented multiple optimization measures for each module model:

Autoencoder threshold setting: a dynamic threshold is used, adjusting the reconstruction error threshold based on real-time data distribution to improve binary classification accuracy and reduce false positives and false negatives[22].

XGBoost confidence threshold: dynamically setting the classification confidence threshold, classifying uncertain samples as unknown attacks to enhance the system's recognition ability for new attacks.

Training epoch adjustment: by tuning the number of training epochs, multiple iterative trainings are conducted to obtain more stable thresholds and loss pa-

rameters, preventing underfitting or overfitting.

Feature selection: excluding timestamp features to avoid model reliance on temporal partition biases, focusing on network behavior features to improve model generalization.

LSTM input optimization: simultaneously feeding forward and backward data into the BiLSTM hidden layer to enhance the model's ability to capture sequential context and alleviate the vanishing gradient problem.

These optimization methods combined ensure accurate detection and classification of various attacks in complex IoT environments.

3.4 Dataset Selection and Preprocessing

In the IoT domain, various datasets suitable for intrusion detection research exist. This project comprehensively considered data quality, attack type coverage, label completeness, and real-world simulation, selecting the two mainstream datasets ToN-IoT and BoT-IoT.

3.4.1 ToN-IoT Dataset

The dataset includes various attack types such as DDoS, DoS, password attacks, scanning attacks, and backdoor attacks, with clearly defined attack labels. It simulates a real IoT network environment and is large and diverse in scale.The dataset maintains a relatively balanced ratio between normal and attack data, and its labeling system is well-structured, facilitating the training of supervised learning models[23]. However, since the "Mitm" attack data is disproportionately low compared to other types of attack data (approximately a 10:1 ratio), such attacks were not sampled.

During preprocessing, invalid columns are removed, missing values filled, and fea-

tures standardized, focusing on network, protocol, and application layer features. Features include IP, ports, protocols, traffic volume, connection status, DNS, SSL/TLS, and HTTP features, ensuring the model can comprehensively analyze multidimensional information.Relevant dataset columns and their meanings:

Field	Meaning	Remarks
src_ip	Source IP address	Host initiating the connection
src_port	Source port number	Local port used by the source
dst_ip	Destination IP address	Host receiving the connection
dst_port	Destination port number	Service port on the destination host
proto	Protocol type	E.g., TCP, UDP
service	Identified application service type	E.g., HTTP, DNS; ”-” denotes unknown
duration	Connection duration (seconds)	Time from start to end of connection
src_bytes	Bytes sent from source	Number of bytes sent by the source
dst_bytes	Bytes sent from destination	Number of bytes sent in response by destination
conn_state	Connection state (as defined by Zeek)	E.g., REJ, S0, S1, SF, OTH

Field	Meaning	Remarks
missed_bytes	Number of missed bytes	Dropped TCP bytes during capture
src_pkts	Number of packets sent by source	Total packets from the source
src_ip_bytes	Total traffic from source IP	Includes TCP/IP headers
dst_pkts	Number of packets sent by destination	Total packets from destination
dst_ip_bytes	Total traffic from destination IP	Includes TCP/IP headers
dns_query	DNS query domain name	
dns_qclass	DNS query class	Typically 1 = IN
dns_qtype	DNS query type	E.g., A, AAAA, MX, CNAME (numeric encoding)
dns_rcode	DNS response code	0 = success
dns_AA	Authoritative answer	Whether the response is authoritative
dns_RD	Recursion desired flag	Whether recursion was requested
dns_RA	Recursion available	Whether the server supports recursion
dns_rejected	DNS query rejected	
ssl_version	TLS/SSL protocol version	E.g., TLSv1.2
ssl_cipher	Cipher suite used	

Field	Meaning	Remarks
ssl_resumed	Resumed session indicator	
ssl_established	TLS session established	Whether the handshake succeeded
ssl_subject	SSL certificate subject	
ssl_issuer	SSL certificate issuer	
http_trans_depth	HTTP transaction depth	E.g., request nesting level
http_method	HTTP request method	E.g., GET, POST
http_uri	Requested URI	
http_version	HTTP protocol version	E.g., HTTP/1.1
http_request_body_len	Request body length	E.g., POST data size
http_response_body_len	Response body length	
http_status_code	HTTP status code	E.g., 200, 404
http_user_agent	Client user agent string	E.g., browser type
http_orig_mime_types	Original MIME types in request	
http_resp_mime_types	MIME types in response	
weird_name	Anomaly type name	E.g., unusual_port
weird_addl	Additional anomaly info	
weird_notice	Whether reported as a notice	
label	Attack type label	
type	Binary label	E.g., attack or normal

Table 3.3: Feature Names and Their Descriptions in the ToN-IoT Dataset

This dataset covers multiple attack types such as DDoS, port scanning, and information theft, with rich feature dimensions including network layer features, traffic statistics, and time-window-related features.The data realistically simulates IoT device network environments, with clear labels, making it suitable for validating model performance in real IoT settings[24].

However, the BoT-IoT dataset still suffers from a severe class imbalance problem, with the ratio of attack data to normal data reaching 99:1. Therefore, in this study, it is used solely for the training, testing, and validation of classification performance in network attack detection (XGBoost module and RNN-BiLSTM module).

For both datasets, an 80% training and 20% testing split is used, strictly separating unknown attack samples to ensure independence and fairness in model training and testing.

3.5 Experimental Setup and Evaluation Metrics

The experimental platform uses the Python programming language, combined with machine learning frameworks such as sklearn and TensorFlow to implement each module model.The hardware environment includes high-performance GPU servers to ensure training efficiency.

In the experimental design, performance evaluation is conducted separately for each module, focusing on four key metrics: Accuracy, Precision, Recall, and F1-score, with the following specific meanings:

1.Accuracy: The proportion of correctly classified samples to the total number of samples, providing a direct assessment of the overall classification capability of the intrusion detection system (IDS) in distinguishing between normal and attack traffic.

2.Precision: The ratio of correctly predicted samples of a specific class to the total number of samples predicted as that class, used to measure the IDS's ability to accurately identify that particular category.

3.Recall: The proportion of correctly identified samples of a specific class by the IDS, reflecting the system's detection coverage and misclassification performance for that category.

4.F1-score: The harmonic mean of Precision and Recall, used to comprehensively evaluate the IDS's balanced performance in reducing false positives and false negatives, providing a more complete measure of the model's classification effectiveness.

Additionally, confusion matrix analysis is used to detail classification results by identifying True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), enabling in-depth exploration of model performance on different attack types and normal samples, guiding model optimization.

Confusion matrix columns and their meanings:

	Predicted Positive	Predicted Negative
Actual Positive(P)	TP (True Positive)	FN (False Negative)
Actual Negative(N)	FP (False Positive)	TN (True Negative)

Table 3.4: Meaning of Values in the Confusion Matrix

Chapter 4

Experiments

4.1 Experimental Objectives

This study builds upon the existing RNN-BiLSTM framework by adding additional inspection modules to enhance the overall performance and functionality of the current IDS system. The main objectives of the experiment include:

1. Enhancing the detection capability of the existing framework: By introducing preprocessing and intermediate processing modules to perform hierarchical handling of network traffic data, the burden on the RNN-BiLSTM module is reduced, thereby improving the overall detection accuracy of the system.

2. Equipping the framework with the ability to detect unknown attacks: By incorporating an XGBoost module, the system can detect zero-day attacks (unknown attack types), providing effective protection for traditional IDS systems when confronting unlabelled or novel attacks.

The experimental objectives are achieved by introducing Autoencoder and XGBoost modules into the existing deep learning architecture, forming a three-layer IDS system capable of enabling hierarchical screening and classification of the entire IoT dataset, ultimately achieving accurate classification of normal traf-

fic, known attacks, and unknown attacks while maintaining high performance in resource-constrained IoT environments.

Experimental limitations:

In this study, the "Reconnaissance" attack type from the BoT-IoT dataset was selected as the unknown attack for subsequent experiments, and the obtained results can serve as a preliminary reference for evaluating the model's capability to detect unknown attacks in the BoT-IoT dataset.

In this study, the "password" attack type from the ToN-IoT dataset was selected as the unknown attack for subsequent experiments, and the obtained results can serve as a preliminary reference for evaluating the model's capability to detect unknown attacks in the ToN-IoT dataset.

4.2 Experimental Setup

The experimental environment uses VirtualBox to construct virtual machines, simulating IoT device settings while taking into account resource-constrained real-world application scenarios.

The virtual machine configurations are listed in the table below:

Name	Resource
CPU	2-core
Virtual Machine Software	virtualBox
Operating System	Ubuntu20.04
Base Memory	1024MB
Python Version	Python 3.10
Installed Software Packages	Pandas, torch(CPU lightweight), joblib, tensorflow, scikitlearn, numpy, xgboost

Table 4.1: Configuration of the Virtual Machine Simulating the IoT Environment

To adapt to resource-constrained IoT device environments and avoid GPU dependency, this study uses the CPU-only version of PyTorch.This configuration

ensures that model training and inference can still be performed under limited computational and memory resources, allowing the system to be deployed on real IoT edge devices[25].

Furthermore, to simulate a real IoT network environment, the experiment loads parsed .pcap files and converts them into .csv format for processing and inference testing.In this way, the network traffic of IoT devices can be reproduced within the virtual machine, ensuring the feasibility and practical applicability of the experimental results.

4.3 Dataset and Preprocessing

This project chooses to use the Bot-IoT and ToN-IoT datasets for training and testing. The characteristics of the two datasets are as follows:

4.3.1 Bot-IoT dataset

Attack type coverage: Includes various types such as DDoS, DoS, Port scanning, and Information Theft.

Data imbalance: The volume of normal data is very small, accounting for approximately 1% of the total, while attack data accounts for 99%.

Feature dimensions: Includes network-layer features, traffic statistics features, and time-window-related features, with clearly defined label fields (Label0: normal, Label1: attack).

4.3.2 ToN-IoT dataset

Rich attack types: Includes a variety of attacks such as Backdoor, DDoS, DoS, Injection, Password, Scanning, Ransomware, and XSS.

Clear data structure: Label fields are well-defined, distinguishing between binary attack/normal classification and specific attack types, suitable for three-layer validation of a complete IDS system.

4.3.3 Data Splitting and Preprocessing

In the experiment, both datasets were processed as follows:

1. Feature Splitting

Name	Value
BoT-IoT dataset	
Attack	0-normal data;1-attack data
Category	Type of attack
Subcategory	Sub-attack type
ToN-IoT dataset	
Label	0-normal data 1-attack data
Type	Specific attack types, such as backdoor

Table 4.2: Target Columns of Each Dataset

2. Excluding Unused Features

Name	Value
BoT-IoT dataset	
pkSeqID	Row Identifier
stime	Record start time
ltime	Record last time
saddr	Source IP address
sport	Source port number
daddr	Destination IP address
dport	Destination port number
ToN-IoT dataset	
src_ip	Source IP address
dst_ip	Destination IP address
http_uri	The requested URI
http_user_agent	Browser or client type

Table 4.3: Excluded Unused Features for Each Dataset

Due to the very small volume of normal data in the Bot-IoT dataset (normal data:attack data 1:99), the AE module is skipped during training and validation on this dataset, and it is directly used to train and test the XGBoost module (known/unknown attack classification) and the RNN-BiLSTM module (specific attack type classification).

3. Training / Testing / Validation Set Division

For each attack type and normal data, 85% is allocated for training and testing, and 15% is used for validating actual performance in a simulated IoT environment. The order of the validation set is adjusted: while retaining "block-like features," attack data is interleaved with normal data to simulate the characteristics of actual IoT network traffic.

4. Feature Processing

Numerical features: Missing values are filled with 0, followed by normalization using MinMaxScaler.

Non-numerical features: Missing values are filled with NaN, followed by encoding using OneHotEncoder.

Further processing for different models:

Autoencoder: Features are directly used for training/detection.

XGBoost: After encoding, split into 80% for training and 20% for testing.

RNN-BiLSTM: Encode and normalize numerical features, perform windowing, with a time step set to 5.

Due to the extremely small amount of normal data in the Bot-IoT dataset, the AE module is skipped, and only XGBoost and RNN-BiLSTM are used for training, testing, and validation; the ToN-IoT dataset is used for full three-layer IDS module validation.

4.4 Experimental Methodology

4.4.1 Module Design and Functionality

Autoencoder module: Used for preliminary screening of the full dataset, dividing data into normal and attack categories.

XGBoost module: Used to further classify attack data, distinguishing between known and unknown attacks.

RNN-BiLSTM module: Classifies the specific types of known attacks and performs secondary detection on any misclassified normal data.

4.4.2 Model Deployment and Optimization Strategies

Pre-trained model transfer: After model training, the weights and preprocessing objects (MinMaxScaler, OneHotEncoder, etc.) are transferred to the virtual machine.

Batch processing strategy: Batch_size = 64 to reduce memory peaks, aligning with IoT data stream processing.

Deferred loading strategy: XGBoost is loaded only when AE identifies an attack, and RNN-BiLSTM is loaded only when XGBoost identifies a known attack.

CPU restriction: TensorFlow is limited to a single thread to prevent full CPU resource consumption.

4.5 Evaluation Metrics

4.5.1 Confusion Matrix Definition

For binary or multi-class classification problems, the elements of the confusion matrix are defined as:

TP_i True Positive for class i

FP_i False Positive for class i

FN_i False Negative for class i

TN_i True Negative for class i

Here, i = 1, 2, 3, ..., C, where C is the total number of classes

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

Recall:

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

F1-score:

$$\text{F1}_i = \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Chapter 5

Results

5.1 Experimental Results

5.1.1 Bot-IoT dataset

The original data structure is:

Attack type	Number
DDOS	1650260
DOS	1650260
Reconnaissance	91082
Normal	477
Theft	79

Table 5.1: Original Data Structure of the Used Bot-IoT Dataset

Data structure after downsampling:

Attack type	Number
DDOS	100000
DOS	100000
Reconnaissance	91082

Table 5.2: Data Structure of the Used Bot-IoT Dataset After Downsampling

The output of XGBoost on the validation set after training is:

Class	Precision	Recall	F1-score	Support
Known	0.99	0.95	0.97	30000
Unknown	0.90	0.97	0.93	13663
Accuracy			0.95	43663
Macro avg	0.94	0.96	0.95	43663
Weighted avg	0.96	0.95	0.96	43663

Table 5.3: XGBoost model Results on the Bot-IoT Dataset

The output of RNN-BiLSTM on the validation set after training is:

Class	Precision	Recall	F1-score	Support
DDos	0.97	1.00	0.98	14438
DoS	0.99	0.99	0.99	14019
Reconnaissance	0.00	0.00	0.00	423
Accuracy			0.98	28880
Macro avg	0.65	0.66	0.66	28880
Weighted avg	0.97	0.98	0.97	28880

Table 5.4: RNN-BiLSTM Model Results on the Bot-IoT Dataset

5.1.2 ToN-IoT Dataset-1

Dataset composition structure:

Data type	Number
Normal	50000
Backdoor	20000
Ddos	20000
Dos	20000
Injection	20000
Password	20000
Scanning	20000
Ransomware	20000
Xss	20000

Table 5.5: Data Structure of the ToN-IoT Dataset-1

Results of training and testing on Dataset-1:

Autoencoder module training and testing results:

Test accuracy: 95%

Attack type	Precision	Recall	F1-score	Support
Normal	0.97	0.81	0.88	42500
Attack	0.94	0.99	0.97	136886
Accuracy			0.95	179386
Macro avg	0.96	0.90	0.92	179386
Weighted avg	0.95	0.95	0.95	179386

Table 5.6: Autoencoder Model Results on the ToN-IoT Dataset-1

XGBoost module training and testing results:

The Password-type attack is set as an unknown attack type

Test accuracy: 96.66%

Attack type	Precision	Recall	F1-score	Support
known	1.00	0.97	0.98	10000
unknown	0.74	0.98	0.84	1000
Accuracy			0.97	11000
Macro avg	0.87	0.97	0.91	11000
Weighted avg	0.97	0.97	0.97	11000

Table 5.7: XGBoost Model Results on the ToN-IoT Dataset-1

Overall IDS validation and testing results:

Test accuracy: 95%

Attack type	Precision	Recall	F1-score	Support
backdoor	1.00	1.00	1.00	3037
ddos	1.00	0.91	0.95	3005
dos	1.00	0.92	0.96	3062
injection	0.99	0.91	0.95	2936
normal	0.97	0.98	0.98	7500
ransomware	0.98	0.99	0.98	2909
scanning	1.00	0.96	0.98	3033
Unknown	0.70	0.98	0.82	2987
xss	1.00	0.81	0.89	3033
Accuracy			0.95	31502
Macro avg	0.96	0.94	0.94	31502
Weighted avg	0.96	0.95	0.95	31502

Table 5.8: Overall IDS Validation Results on the ToN-IoT Dataset-1

Confusion matrix:

	backdoor	ddos	dos	injection	normal	ransomware	scanning	Unknown	xss
backdoor	3024	0	3	0	1	6	0	3	0
ddos	0	2747	0	0	35	2	1	217	3
dos	0	0	2822	0	155	0	3	82	0
injection	0	0	0	2659	5	0	0	221	2
normal	1	2	0	0	7374	57	7	51	0
ransomware	0	0	0	4	27	2874	0	3	0
scanning	1	0	0	0	3	3	2924	102	0
Unknown	0	4	0	26	4	0	0	2933	0
xss	0	0	0	0	5	1	1	575	2451

Table 5.9: Confusion Matrix of the Overall IDS Validation on the ToN-IoT Dataset-1

Relevant performance metrics:

Metric type	Number
Total number of samples in validation dataset	31502
Total inference time	231.8135 seconds
Average inference latency	7.3587 ms/sample
Throughput	135.89 sample/sec
Memory usage	434.54 MB
CPU utilization	3.10%

Table 5.10: Performance Metrics of the Overall IDS Validation on the ToN-IoT Dataset-1

5.1.3 ToN-IoT Dataset-2

Dataset composition structure:

Data Type	Number
Normal	62187
Backdoor	20000
Ddos	20000
Dos	20000
Injection	20000
Password	20000
Scanning	20000
Ransomware	20000
Xss	20000

Table 5.11: Data Structure of the ToN-IoT Dataset-2

Results of training and testing on Dataset-2:

Autoencoder module training and testing results:

Test accuracy: 93%

Type	Precision	Recall	F1-score	Support
Normal	0.92	0.84	0.88	52859
Attack	0.94	0.97	0.95	135999
Accuracy			0.93	188858
Macro avg	0.93	0.90	0.92	188858
Weighted avg	0.93	0.93	0.93	188858

Table 5.12: Autoencoder Model Results on the ToN-IoT Dataset-2

XGBoost module training and testing results:

The Password-type attack is set as an unknown attack type

Test accuracy: 92.66%

Type	Precision	Recall	F1-score	Support
known	0.97	0.94	0.96	10000
unknown	0.57	0.75	0.65	1000
Accuracy			0.93	11000
Macro avg	0.77	0.85	0.80	11000
Weighted avg	0.94	0.93	0.93	11000

Table 5.13: XGBoost Model Results on the ToN-IoT Dataset-2

Overall IDS validation and testing results:

Test accuracy: 92%

Type	Precision	Recall	F1-score	Support
backdoor	0.99	0.77	0.86	3024
ddos	0.99	0.87	0.93	3086
dos	1.00	0.95	0.97	3047
injection	1.00	0.90	0.95	3014
normal	0.93	0.99	0.96	9328
ransomware	1.00	1.00	1.00	2936
scanning	1.00	0.91	0.95	3016
unknown	0.58	0.75	0.65	2922
xss	0.99	0.89	0.94	2956
Accuracy			0.92	33329
Macro avg	0.94	0.89	0.91	33329
Weighted avg	0.94	0.91	0.92	33329

Table 5.14: Overall IDS Validation Results on the ToN-IoT Dataset-2

Confusion matrix:

	backdoor	ddos	dos	injection	normal	ransomware	scanning	unknown	xss
backdoor	2321	0	3	0	423	0	0	277	0
ddos	0	2672	0	0	51	0	0	346	0
dos	0	0	2894	0	8	0	3	142	0
injection	33	3	0	2725	29	0	0	257	0
normal	0	0	0	0	9189	4	5	75	3
ransomware	0	0	0	3	3	2930	0	0	0
scanning	2	0	0	0	96	0	2730	187	0
unknown	0	13	0	0	82	0	0	2195	2
xss	0	2	0	0	14	0	4	305	2631

Table 5.15: Confusion Matrix of the Overall IDS Validation on the ToN-IoT Dataset-2

Relevant performance metrics:

Metric type	Number
Total number of samples in validation dataset	33329
Total inference time	247.8233 seconds
Average inference latency	7.4177 ms/sample
Throughput	134.83 sample/sec
Memory usage	425.86 MB
CPU utilization	1.50%

Table 5.16: Performance Metrics of the Overall IDS Validation on the ToN-IoT Dataset-2

5.2 Results Analysis

AE Module Performance

On the ToN-IoT dataset, the validation accuracies in two trials were 95% and 93%, with Precision/Recall for attack data exceeding 0.94. Classification of normal data shows slight deviation, but the primary function is met: reducing the workload for subsequent modules and enhancing overall system efficiency

XGBoost Module Performance

On the Bot-IoT dataset, accuracy is 95%, with Precision 0.99 and Recall 0.95 for known attacks, and Precision 0.90 and Recall 0.97 for unknown attacks.On the ToN-IoT dataset, accuracies were 92% and 96%, demonstrating that the module can identify most known attacks and possesses unknown attack detection capability.

Overall IDS Framework Performance

High precision and recall, significant unknown attack detection capability, low false positives, average inference latency 7.367–7.42 ms/sample, throughput approximately 134–136 samples/second, low CPU usage (1.5%–3%), and memory usage under 450 MB.Suitable for deployment on resource-constrained IoT devices, with the option to selectively disable modules to optimize performance according

to the scenario.

Attack Type Analysis

For payload-feature attacks such as Injection and XSS, recognition performance is slightly lower, requiring further optimization of feature extraction methods.Typical attacks such as Backdoor, DDoS, and DoS are recognized with high accuracy, making the system suitable for large-scale network protection scenarios.

Summary of Findings

The three-layer IDS system (AE → XGBoost → RNN-BiLSTM) achieves hierarchical detection of normal traffic, known attacks, and unknown attacks. The system performs well on both Bot-IoT and ToN-IoT datasets, with an accuracy of 92%–95% and notable capability for detecting unknown attacks. The system exhibits low average latency, high throughput, and low resource usage, making it suitable for deployment on IoT edge devices. The The modular design enables selective module disabling to balance performance and resource utilization. according to requirements, balancing performance and resource utilization. There is room for improvement in detecting payload-based attacks, such as Injection and XSS.

5.3 Analysis and Discussion

This study focuses on intrusion detection systems (IDS) in IoT environments and proposes a three-tier modular architecture that integrates Autoencoder, XGBoost, and RNN-BiLSTM, building upon the original RNN-BiLSTM framework. The core objectives of this architecture are as follows:

Enhancing detection capability: employing a multi-module hierarchical structure to reduce false positives and improve detection accuracy.

Unknown attack detection: addressing the limitation of conventional deep learning-

based IDS in handling zero-day attacks.

Adaptability to IoT edge environments: ensuring acceptable throughput and latency under constrained resources.

Through experiments, this study not only validates the effectiveness of the framework on the Bot-IoT and ToN-IoT datasets but also demonstrates its feasibility and practical value for deployment in resource-constrained IoT environments.

5.3.1 Performance Analysis of Each Module

Performance and Discussion of the Autoencoder Module

Experimental results on the ToN-IoT dataset show that the AE module achieved accuracies of 95% and 93%, with both precision and recall for attack detection exceeding 0.94, demonstrating its high effectiveness in the initial filtering stage.

The advantages of this module include:

Rapid filtering of attack data: reducing the computational load of subsequent models and improving overall efficiency.

Strong generalization capability: distinguishing between normal and abnormal traffic without explicitly relying on attack-specific features.

However, this module also presents certain limitations:

Bias in recognizing normal traffic: some normal traffic was misclassified as attacks, indicating that the feature reconstruction mechanism of the AE may still be overly sensitive, leading to a certain level of false positives that require correction in subsequent layers.

Ineffectiveness on the Bot-IoT dataset: due to the extremely low proportion of normal data (approximately 1%), the AE module could not function effectively on this dataset, preventing the system from performing complete three-layer validation.

Therefore, the AE module is more suitable for datasets with relatively balanced

distributions of normal and attack data, or for deployment in real IoT environments in combination with data augmentation and synthetic normal traffic generation.

Performance and Discussion of the XGBoost Module

The XGBoost module is primarily designed to distinguish between known and unknown attacks. The experimental results show that:

On the Bot-IoT dataset, it achieved an overall accuracy of 95%, with precision and recall for known attacks of 0.99 and 0.95, respectively; and precision and recall for unknown attacks of 0.90 and 0.97.

On the ToN-IoT dataset, accuracies of 92% and 96% were achieved. Notably, the module maintained high detection performance even when the Password attack type was designated as an unknown attack.

These results suggest that XGBoost possesses strong generalization capability, enabling effective detection of zero-day attacks. The main reasons include:

Flexibility of tree-based decision boundaries: enabling effective handling of high-dimensional and sparse features.

High adaptability: requiring no large-scale pretraining, making it suitable for resource-constrained IoT environments.

However, certain limitations remain:

Instability in unknown attack detection: in the second ToN-IoT experiment, the precision for unknown attacks dropped to 0.57, with an F1-score of only 0.65. This indicates that when attack feature distributions are complex, the model may struggle to capture the latent patterns of unknown attacks.

Dependence on feature engineering: effective performance requires high-quality feature selection and preprocessing; otherwise, model performance deteriorates significantly.

The experimental scope is limited: in the BOT-IoT dataset, the “Reconnaissance”

attack type was selected as the unknown attack; in the ToN-IoT dataset, the "password" attack type was chosen as the unknown attack. Therefore, the range of experimental testing remains limited, but the results still demonstrate the model's capability to detect unknown attacks.

Therefore, within the system, the XGBoost module functions as a "gatekeeper." However, its stability in detecting unknown attacks still requires improvement, potentially through enhanced feature extraction and the integration of semi-supervised learning methods.

Performance and Discussion of the RNN-BiLSTM Module

The RNN-BiLSTM module is employed for fine-grained classification of known attacks. The experimental results demonstrate that:

On the Bot-IoT dataset, the overall accuracy reached 98%, with nearly perfect classification performance for DDoS and DoS attacks (recall ¿ 0.99).

On the ToN-IoT dataset, the overall IDS accuracy ranged from 92% to 95%, with most attack categories achieving precision and recall close to or above 0.95.

These results indicate that:

Outstanding temporal sequence modeling capability of BiLSTM: it can effectively capture sequential patterns in network traffic, enabling accurate differentiation of typical attack behaviors. High effectiveness in identifying mainstream attacks: the module achieves very high accuracy for attacks such as DDoS, DoS, and Backdoor.

However, some limitations remain:

Insufficient detection of payload-based attacks (e.g., Injection, XSS): the recall is noticeably low, suggesting that the RNN-BiLSTM struggles to capture sufficient patterns in complex application-layer attacks.

High data dependency: when attack samples are scarce or feature distributions are imbalanced, the classification performance drops significantly (e.g., recall for

Reconnaissance attacks in Bot-IoT = 0).
This suggests that while the RNN-BiLSTM module possesses strong temporal modeling capabilities, its performance in diverse attack scenarios still requires enhancement through advanced feature engineering or the integration of multi-modal data (e.g., combining system logs with traffic features).

5.4 Overall System Performance Analysis

The overall IDS system demonstrated stable performance on both the Bot-IoT and ToN-IoT datasets, maintaining an accuracy between 92% and 95%. Moreover, it sustained low latency and low resource consumption even under resource-constrained IoT environments: Latency: The average inference latency ranged from 7.36 to 7.42 ms per sample, which is sufficient to meet the real-time detection requirements of most IoT scenarios. Throughput: The system sustained a processing rate of 134–136 samples per second, demonstrating its ability to continuously handle high-frequency IoT traffic. Resource consumption: Memory usage remained below 450 MB, and CPU utilization was under 3%, making the framework suitable for IoT edge devices with limited resources. These results indicate that the proposed three-layer IDS framework achieves a favorable balance between performance and resource consumption, confirming its feasibility for real-world deployment.

5.5 Comparison with Existing Studies

Compared with traditional IDS and approaches that rely solely on conventional machine learning models, this study has the following advantages:
Firstly, the hierarchical detection mechanism composed of multiple modules enables the model in this study to achieve superior detection capabilities for both

known and unknown attacks compared to a single model, while also providing a certain degree of generalization.

Secondly, this study demonstrates strong resource adaptability. The model maintains high detection accuracy and throughput even in CPU-only environments, whereas some existing algorithms and models rely on GPUs to enhance performance, which may pose a limitation in resource-constrained IoT device settings.

Thirdly, this study demonstrates significant performance in unknown attack detection. By incorporating the XGBoost model, it effectively addresses the limitations of traditional IDS in handling zero-day attacks.

However, it should be noted that, compared to some of the latest IDS studies based on Graph Neural Networks (GNN) or federated learning, this study still has certain limitations in terms of attack diversity and cross-domain generalization capability.For example, GNN models can more accurately capture relationships between network traffic, whereas the model in this study primarily relies on sequential features and feature engineering.

5.6 Practical Application Potential and Limitations

Potential:

Suitable for edge computing environments: Low resource consumption enables deployment on IoT endpoints and edge gateways.

High flexibility: Modules can be selectively disabled according to application scenarios. For example, deploying only AE+XGBoost for rapid unknown attack detection, or XGBoost+RNN-BiLSTM for multi-class attack classification.

High accuracy and low latency: Meets the dual requirements of real-time performance and reliability in smart homes, industrial IoT, and similar applications.

Limitations:

Limited detection of complex payload-based attacks: Recall for Injection and XSS attacks is relatively low.

Dependence on feature engineering: Effective training requires relatively balanced attack type datasets to ensure the model learns sufficient attack features and categories. Additionally, training the RNN-BiLSTM relies on data being organized in sequential blocks by attack type.

Limited cross-dataset generalization: Performance may fluctuate when transferring the model across different IoT datasets.

5.7 Future Improvement Directions

To address the aforementioned limitations, future research can focus on the following directions:

Data augmentation and few-shot learning:

Employ generative adversarial networks (GANs) and few-shot learning techniques to improve detection of attacks with limited samples.

Multimodal data fusion: Integrate traffic data, device logs, and user behavior features to generate enriched representations, enhancing detection capability in complex attack scenarios.

Enhancing generalization: Train on larger and more diverse datasets and validate across different datasets to improve cross-domain generalization. Lightweight model optimization: Further compress and quantize the model to enhance usability on ultra-low-power IoT devices.

Improving interpretability: Incorporate explainable AI (XAI) methods to assist operators in understanding IDS decision-making, increasing practical trustworthiness.

5.8 Chapter Summary

From the analysis and discussion, the following conclusions can be drawn:

The three-layer IDS framework demonstrates excellent performance in terms of accuracy, real-time processing, and resource usage, indicating its feasibility for deployment on IoT edge devices.

The AE module effectively reduces system computational overhead, but its performance is limited under highly imbalanced data distributions. It is applicable only when normal and attack data are distinguishable, and can be skipped if no normal data is available.

The XGBoost module shows strong capability in detecting unknown attacks, although its detection stability requires further optimization.

The RNN-BiLSTM module performs well on common attack types but shows limitations in detecting complex payload attacks. Moreover, due to its windowed data training process, it requires training data to be partially ordered by attack type to effectively learn attack patterns.

In summary, this study provides an IoT intrusion detection solution that balances performance, resource adaptability, and unknown attack detection capability, though further improvements are needed in generalization and complex payload attack detection.

Chapter 6

Conclusion

In recent years, with the rapid development of the Internet of Things (IoT), IoT devices have been increasingly applied across various domains. Consequently, the cybersecurity issues of these devices have become a critical focus in the field of network security research.Given that IoT devices are inherently resource-constrained and require high real-time performance, while also facing unknown attack threats similar to other network environments, achieving efficient intrusion detection under lightweight constraints has become a key challenge in current research. In this context, there is an urgent need for an IDS model that combines the following characteristics: a lightweight design suitable for IoT edge devices to reduce device burden, effective detection of unknown attacks, and full utilization of the advantages of multi-model integration to leverage each model's strengths in classification and sequential data processing. To address this need, this study proposes a multi-model hybrid intrusion detection framework. The framework employs a three-layer structural design, reducing the data processing load layer by layer, thereby enhancing detection capability while maintaining processing efficiency.Specifically, the framework incorporates an XGBoost model combined with dynamic threshold calculation to effectively identify unknown attacks, while

retaining the high accuracy of the RNN-BiLSTM model in sequential data processing and classification, thus balancing the detection of known attacks with the identification of unknown attacks.

Moreover, the framework is designed to be lightweight, meeting the fast data processing requirements of IoT devices while fully considering the resource-constrained characteristics of the devices, ensuring a low deployment burden.Experimental results indicate that, on the BoT-IoT dataset, the model achieves 95% accuracy in detecting unknown attacks and maintains excellent performance for known attack detection (precision of 0.99 and recall of 0.95).On two subsets of the ToN-IoT dataset, the detection accuracy for unknown attacks reaches 97% and 93%, respectively, fully demonstrating that the framework successfully enhances unknown attack detection by adding additional modules on top of the original models.Meanwhile, the model preserves its original multi-class classification capability and classification accuracy.

In the experimental and analysis phase, this study utilized multiple datasets to thoroughly test the model under simulated resource-constrained IoT device environments and provided a feasible reproducibility framework.Experimental results demonstrate that this multi-model hybrid framework, while maintaining lightweight design and high efficiency, effectively handles unknown attacks, providing a feasible and efficient technical solution for intrusion detection in IoT environments.

Regarding future research directions, firstly, larger and more realistic IoT datasets are needed for training, testing, and validation.This approach can more accurately reflect the model's functional characteristics, uncover potential issues, and further optimize the model to better adapt to real-world application environments.Secondly, the IDS model should be deployed on actual IoT devices to conduct live testing.In real operational environments, data characteristics and

real-time processing requirements are more complex. Although this study simulated IoT device environments using virtual machines, it cannot fully reproduce the challenges present in real-world scenarios, particularly in real-time data processing.Therefore, only through validation and debugging on actual IoT device environments can it be ensured that the model can be stably and accurately applied in real-world scenarios.

Furthermore, more model construction schemes should be explored and compared to identify the most suitable approach for this framework.By exploring efficient lightweight strategies and optimization schemes, the model's practicality and adaptability in resource-constrained environments can be enhanced.At the same time, the impact of cross-domain deployment and heterogeneous device environments on model performance should be considered, ultimately enabling the model to be extended to broader application scenarios.

In summary, this study constructed a three-layer lightweight IDS framework through multi-model integration, enhancing the detection capability for unknown attacks while preserving the multi-class classification accuracy of the original models.Validated through experiments on multiple datasets, the framework not only meets the fast data processing requirements of IoT devices but also fully considers resource constraints, providing an effective technical approach and practical reference for intrusion detection research in IoT environments.In the future, combining larger-scale real-world data and live device validation, along with further optimization of the model construction scheme, will further enhance the framework's adaptability and scalability, providing a more solid foundation for IoT cybersecurity protection.

References

[1] P. Dhull, A. P. Guevara, M. Ansari, S. Pollin, N. Shariati, and D. Schreurs, "Internet of things networks: Enabling simultaneous wireless information and power transfer," *IEEE Microwave Magazine*, vol. 23, no. 3, pp. 39–54, 2022. 1

[2] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "Ddos in the iot: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017. 2

[3] J. Cannady, J. Harrell *et al.*, "A comparative analysis of current intrusion detection technologies," in *Proceedings of the Fourth Technology for Information Security Conference*, vol. 96, 1996. 2

[4] A. Heidari and M. A. J. Jamali, "Internet of things intrusion detection systems: A comprehensive review and future directions," *Cluster Computing*, vol. 26, no. 6, pp. 3753–3780, 2023. [Online]. Available: https://doi.org/10.1007/s10586-022-03776-z 3

[5] M. H. Miraz, M. Ali, P. S. Excell, and R. Picking, "Internet of nano-things, things and everything: Future growth trends," *Future Internet*, vol. 10, no. 8, p. 68, 2018. [Online]. Available: https://www.mdpi.com/1999-5903/10/8/68 6

[6] N. Srivastava and P. Pandey, "Internet of things (iot): Applications, trends,

issues and challenges," *Materials Today: Proceedings*, vol. 69, pp. 587–591, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214785322063209 6

[7] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for iot security based on learning techniques," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2671–2701, 2019. 6

[8] A. López-Vargas, M. Fuentes, and M. Vivar, "Challenges and opportunities of the internet of things for global development to achieve the united nations sustainable development goals," *IEEE Access*, vol. 8, pp. 37 202–37 213, 2020. 7

[9] M. binti Mohamad Noor and W. H. Hassan, "Current research on internet of things (iot) security: A survey," *Computer Networks*, vol. 148, pp. 283–294, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128618307035 7

[10] C. Ioannou and V. Vassiliou, "Network attack classification in iot using support vector machines," *Journal of Sensor and Actuator Networks*, vol. 10, no. 3, p. 58, 2021. [Online]. Available: https://www.mdpi.com/2224-2708/10/3/58 8

[11] M. A. A. da Cruz, L. R. Abbade, P. Lorenz, S. B. Mafra, and J. J. P. C. Rodrigues, "Detecting compromised iot devices through xgboost," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 12, pp. 15 392–15 399, 2023. 8

[12] S. J. Nhlapo and M. N. W. Nkongolo, "Zero-day attack and ransomware detection," *arXiv preprint arXiv:2408.05244*, 2024. [Online]. Available: https://arxiv.org/abs/2408.05244 8

[13] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, "Utilising deep learning techniques for effective zero-day attack detection," *Electronics*, vol. 9, no. 10, p. 1684, 2020. [Online]. Available: https://www.mdpi.com/2079-9292/9/10/1684 9, 15

[14] U. Zahoora, M. Rajarajan, Z. Pan, and A. Khan, "Zero-day ransomware attack detection using deep contractive autoencoder and voting based ensemble classifier," *Applied Intelligence*, vol. 52, no. 12, pp. 13 941–13 960, 2022. [Online]. Available: https://doi.org/10.1007/s10489-022-03244-6 9

[15] A. Aldaej, T. A. Ahanger, and I. Ullah, "Deep learning-inspired iot-ids mechanism for edge computing environments," *Sensors*, vol. 23, no. 24, p. 9869, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/24/9869 10

[16] A. Alqahtani, "Optimized deep autoencoder and bilstm for intrusion detection in iots-fog computing," *Multimedia Tools and Applications*, vol. 84, no. 8, pp. 4907–4943, 2025. [Online]. Available: https://doi.org/10.1007/s11042-024-18919-0 10

[17] K. Berahmand, F. Daneshfar, E. S. Salehi, Y. Li, and Y. Xu, "Autoencoders and their applications in machine learning: a survey," *Artificial Intelligence Review*, vol. 57, no. 2, p. 28, 2024. [Online]. Available: https://doi.org/10.1007/s10462-023-10662-6 15

[18] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 785–794. [Online]. Available: https://doi.org/10.1145/2939672.2939785 17

[19] P. Zhang, Y. Jia, and Y. Shang, "Research and application of xgboost in imbalanced data," *International Journal of Distributed Sensor Networks*, vol. 18, no. 6, 2022. 17

[20] W. Fang, Y. Chen, and Q. Xue, "Survey on research of rnn-based spatio-temporal sequence prediction algorithms," *Journal on Big Data*, vol. 3, no. 3, p. 97, 2021. 18

[21] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "The performance of lstm and bilstm in forecasting time series," in *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 3285–3292. 19

[22] J. Clark, Z. Liu, and N. Japkowicz, "Adaptive threshold for outlier detection on data streams," in *2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA)*, 2018, pp. 41–49. 19

[23] I. Tareq, B. M. Elbagoury, S. El-Regaily, and E.-S. M. El-Horbaty, "Analysis of ton-iot, unw-nb15, and edge-iiot datasets using dl in cybersecurity for iot," *Applied Sciences*, vol. 12, no. 19, p. 9572, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/19/9572 20

[24] J. M. Peterson, J. L. Leevy, and T. M. Khoshgoftaar, "A review and analysis of the bot-iot dataset," in *2021 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2021, pp. 20–27. 24

[25] T. Wang, T. Shi, X. Liu, J. Wang, B. Liu, Y. Li, and Y. She, "Minimizing latency for multi-dnn inference on resource-limited cpu-only edge devices," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 2239–2248. 28

Use of Generative AI:

In the preparation of this thesis, GenAI tools were employed solely for language refinement. The tools provided suggestions regarding academic vocabulary and style; however, all selections, revisions, and decisions about incorporation were made independently by the author. No generative AI tools were used to produce research content, data analysis, or original contributions.

Code Repository

The complete source code for this thesis is available at:

`https://github.com/liubolun1997/bolunliu_finalproject/tree/master/final_project`